

24CS102 — SOFTWARE DEVELOPMENT PRACTICES

UNIT I: AGILE SOFTWARE DEVELOPMENT, Git AND GitHub

Course Study Handbook & University Exam Preparation Guide | Sem - 1

COURSE MATRIX & SYLLABUS ALIGNMENT

Course Objectives: Understand the essence of agile development methods; setup and manage GitHub repositories.

Target Course Outcomes (COs):

- **CO1 (K2):** Understand basic software engineering practices effectively.
- **CO2 (K3):** Apply version control using Git and GitHub, and manage code repositories proficiently.

Prescribed Textbooks: Roger S. Pressman (9th Ed., 2020) | Scott Chacon & Ben Straub, *Pro GIT* (3rd Ed., 2014).

1. SOFTWARE ENGINEERING PRACTICES & SDLC PROCESS MODELS

1.1 Definition & Nature of Software Engineering

According to the **IEEE Standard Definition**, Software Engineering is defined as: *'The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.'*

Key Characteristics of Software as a Product:

1. Software is developed or engineered; it is not manufactured in the classical sense.
2. Software does not 'wear out' over time, though it suffers from deterioration due to environmental changes.
3. Although the industry is moving toward component-based assembly, most software continues to be custom built.

CORE CONCEPT: SOFTWARE PROCESS ACTIVITIES

Software Process Definition: A structured set of core activities required to develop a software system:

1. **Specification:** Defining what the system should do.
2. **Design & Implementation:** Defining the architecture and writing code.
3. **Validation:** Ensuring software satisfies customer expectations.
4. **Evolution:** Modifying software to meet changing user needs.

1.2 Verification vs. Validation

Dimension	Verification	Validation
Core Question	Are we building the product right?	Are we building the right product?
Focus	Confirms software correctly implements specific functionality.	Ensures software satisfies actual customer requirements.
Activities	Reviews, inspections, walkthroughs, static analysis.	Functional testing, user acceptance testing (UAT), system testing.
Execution Point	Performed throughout early development phases.	Performed post-implementation on executable increments.

1.3 General Principles of Software Engineering (Pressman)

- 1. The First Principle — The Reason It All Exists:** Does this add real value to the system? If no, don't do it.
- 2. Keep It Simple (KISS):** All design should be as simple as possible, but no simpler.
- 3. Maintain the Vision:** A clear vision is essential to success; avoid architecture drift.
- 4. What You Produce, Others Will Consume:** Design and code for maintainability and readability.
- 5. Be Open to the Future:** Never design yourself into a corner; prepare for changing environments.
- 6. Plan Ahead for Reuse:** Reuse saves time, reduces cost, and improves reliability.
- 7. The Seventh Principle — Think!:** Placing clear, complete thought before action produces better results.

1.4 The Waterfall Model (Linear-Sequential Life Cycle)

The **Waterfall Model** is a classic SDLC process model where development flows sequentially from one phase to the next. The output of one phase forms the direct input for the subsequent phase without phase overlap.

Phase	Core Deliverable / Activity Description
1. Requirements Gathering	All system requirements are captured and documented in an SRS (Software Requirements Specification).
2. System Design	Requirements are analyzed to define hardware/software specs and overall system architecture.
3. Implementation	System is coded in small programs called units; each unit undergoes unit testing.
4. Integration & Testing	All units are integrated into a complete system and tested for faults and failures.
5. Deployment	Tested software is released to the client environment or market.
6. Maintenance	Patches and enhanced versions are deployed to fix live client issues and enhance capabilities.

When to Apply Waterfall: Well-understood, clear, and fixed requirements; stable product definition; understood technology stack; short project duration with ample resource expertise.

Deficiencies: No working software is produced until late in the lifecycle; high risk & uncertainty; late requirement changes cause catastrophic cost overruns.

2. AGILITY PRINCIPLES & COST OF CHANGE

2.1 Agility & The Agile Manifesto

Agility represents the ability to move quickly and easily—a nimble team's capacity to respond rapidly and effectively to change (changes in software requirements, team composition, or underlying technology).

AGILE MANIFESTO FOUNDATION

THE AGILE MANIFESTO (2001) FOUR CORE VALUES:

1. **Individuals and interactions** over processes and tools.
2. **Working software** over comprehensive documentation.
3. **Customer collaboration** over contract negotiation.
4. **Responding to change** over following a plan.

Note: While there is value in the items on the right, we value the items on the left more.

2.2 Agility and the Cost of Change Curve

In traditional linear software models (Waterfall), the cost of accommodating changes increases **nonlinearly** as a project progresses due to architectural rework, re-coding, and re-testing. A well-designed agile process **'flattens'** this curve by incorporating short iterative cycles, continuous integration, customer feedback, and daily refactoring, allowing teams to accommodate late changes efficiently.

THE FLATTENED COST OF CHANGE CURVE

Visual Concept: Cost of Change Dynamics

- **Traditional (Waterfall) Curve:** Exponential cost trajectory as project advances through Design -> Code -> Integration -> Testing -> Deployment.
- **Agile 'Flattened' Curve:** Near-linear cost trajectory maintained across late project stages by continuously proving design via working increments and refactoring.

2.3 Core Agility Principles (12 Principles Summary)

- **Continuous Delivery:** Highest priority is customer satisfaction via early and continuous delivery of valuable software.
- **Welcome Change:** Harness changing requirements, even late in development, for customer competitive advantage.
- **Frequent Increments:** Deliver working software frequently (weeks to months, preferring shorter timescales).
- **Daily Collaboration:** Business stakeholders and developers must work together daily throughout the project.
- **Motivated Individuals:** Build projects around motivated people; give them support and trust to get the job done.
- **Face-to-Face Conversation:** Most efficient and effective method of conveying information within a team.
- **Working Software:** Primary measure of progress.
- **Self-Organizing Teams:** Best architectures, requirements, and designs emerge from self-organizing teams.

3. EXTREME PROGRAMMING (XP) & INDUSTRIAL XP (IXP)

3.1 The 5 Core XP Values

Value	Operational Practice & Meaning
Communication	Emphasizes close verbal collaboration between developers & customers; avoids voluminous documentation.
Simplicity	Restricts design strictly to immediate needs ('YAGNI' - You Aren't Gonna Need It); refactors later if required.
Feedback	Derived continuously from 3 sources: automated unit tests, customer acceptance tests, and rapid estimates.
Courage	Discipline to design for today, accepting that future requirements may alter current implementations.
Respect	Mutual respect built across the team as successful working software increments are delivered.

3.2 Extreme Programming Process Activities & Practices

- 1. Planning Game & User Stories:** Requirements are written by customers on index cards as *User Stories*. Priority and business value are assigned. Stories are scheduled based on risk and value.
- 2. Project Velocity:** Measured as the number of customer stories implemented during the first release; used to accurately estimate subsequent release schedules.
- 3. Pair Programming:** Two developers work together at one workstation (one driver, one navigator) to continuously review code and enhance quality.
- 4. Test-Driven Development (TDD):** Automated unit tests are created *before* code is written. Code must pass all tests before integration.
- 5. Refactoring:** Continuous restructuring of code design without altering external behavior to maintain simplicity.

3.3 Industrial XP (IXP) Enterprise Extensions

IXP refines traditional XP to suit large enterprise organizations by introducing **6 new management practices**:

- **Readiness Assessment:** Evaluates if organizational culture, environment, and stakeholders support agile values.
- **Project Community:** Explicitly expands team roles to include legal staff, auditors, and sales personnel.
- **Project Chartering:** Assesses business justification and organizational alignment before kickoff.
- **Test-Driven Management:** Establishes measurable destinations and criteria to evaluate project health.
- **Retrospectives:** Post-increment reviews to capture lessons learned and continuously tune processes.
- **Continuous Learning:** Encourages ongoing skill acquisition and team adaptation.

4. AGILE PROCESS MODELS FRAMEWORKS

Agile Model	Core Philosophy & Defining Mechanics
Scrum	Focuses on iterative project management using Sprints (30-day time-boxes), Product/Sprint Backlogs, Daily 15-min Scrum meetings, and Scrum Master protection.
Adaptive Software Development (ASD)	Embraces continuous change through a 3-phase cycle: Speculation (adaptive planning) -> Collaboration -> Learning.

Agile Model	Core Philosophy & Defining Mechanics
Dynamic Systems Development Method (DSDM)	Governed by the 80% Rule (80% value delivered in 20% effort). Iterative cycles: Feasibility, Business Study, Functional Model, Design/Build, Implementation.
Crystal Family	Tailored by team size & criticality: Crystal Clear (<=8 people, lightweight), Crystal Yellow (10-20 people, moderate formality), Crystal Orange (20-50 people, enterprise risk).
Feature Driven Development (FDD)	Emphasizes feature-based decomposition (the). 5 Activities; 6 Milestones per feature within 2-week iterations.
Lean Software Development (LSD)	Inspired by Lean manufacturing. 7 Principles: Eliminate waste, build quality in, create knowledge, defer commitment, deliver fast, respect people, optimize whole.
Agile Modeling (AM)	Collection of principles for lightweight modeling: Model with purpose, use multiple models, content over representation, discard unnecessary models.
Agile Unified Process (AUP)	Adopts 'serial in the large' and 'iterative in the small'. Activities: Modeling (just barely good enough), Implementation, Testing, Deployment, Config/PM.

4.1 Scrum Scaling: Scaling Up vs. Scaling Out

- **Scaling Up:** Applying Scrum to large software systems requiring multi-team coordination, up-front architecture design, cross-team sync meetings, and regular system integration builds.
- **Scaling Out:** Introducing Scrum across a traditional, bureaucratic organization by adapting quality procedures and training multi-skilled personnel.

5. VERSION CONTROL & GIT FUNDAMENTALS

5.1 Version Control Architecture Types

1. **Local VCS:** Keeps single database on local computer tracking file revisions (e.g., RCS). Vulnerable to single disk failure.
2. **Centralized VCS (CVCS):** Central server contains all versioned files; clients check out files (e.g., SVN, CVS). Single point of server failure.
3. **Distributed VCS (DVCS):** Clients fully mirror the entire repository including full commit history (e.g., Git, Mercurial). No single point of failure.

CORE GIT ARCHITECTURE THE THREE STATES OF GIT:

1. **Committed:** Data safely stored in your local Git repository database (`.git`).
 2. **Modified:** File has been changed in the working directory but not yet staged.
 3. **Staged:** Marked a modified file in its current version to go into your next commit snapshot.
- Corresponding Sections:** Working Directory -> Staging Area (Index) -> Git Directory (Repository)

5.2 Essential Git Commands & Workflows

```
# Initialize a new repository in current directory
$ git init

# Track files and stage changes for commit
$ git add filename.ext
$ git add .

# Commit staged changes with descriptive message
$ git commit -m 'Initial commit of project files'

# View file status (staged, modified, untracked)
$ git status

# View commit history log
$ git log --oneline --graph --all
```

5.3 Undoing Changes & Tagging

- **Unstage a File:** `git restore --staged <file>` (or `git reset HEAD <file>`).
- **Discard Working Directory Changes:** `git restore <file>` (or `git checkout -- <file>`).
- **Amend Last Commit:** `git commit --amend -m 'Updated message'`.
- **Lightweight Tags:** Simple pointer to specific commit: `git tag v1.0`.
- **Annotated Tags:** Stored as full objects in Git DB with tagger details & message: `git tag -a v1.4 -m 'Version 1.4 release'`.

6. GIT BRANCHING, MERGING & REBASING

6.1 Branching Concepts & Management

A branch in Git is a lightweight, movable pointer to a commit snapshot. The default branch is typically main (formerly master).

```
# Create and switch to new branch simultaneously
$ git checkout -b feature-iss53
$ git switch -c feature-iss53

# Branch Management & Inspection
$ git branch --merged # List branches merged into current branch
$ git branch --no-merged # List unmerged active branches
$ git branch -d feature # Delete merged branch safely
$ git branch -D feature # Force delete unmerged branch
$ git branch -m old-name new-name # Rename local branch
```

6.2 Deep Dive: Git Merge vs. Git Rebase

Feature	Git Merge	Git Rebase
History Structure	Preserves original non-linear commit graph including explicit merge commits.	Rewrites commit history by moving commits onto a new base, producing a linear graph.
Mechanics	Creates a 3-way merge commit combining snapshots from two tip commits.	Replays individual commits one-by-one onto target base, creating new commit hashes.
Destructiveness	Non-destructive; existing commit history remains untouched.	Destructive; alters history and generates new commit checksums.
Conflict Resolution	Resolved once in the single merge commit.	Conflicts resolved repeatedly for each replayed commit.
Best Use Case	Integrating completed feature branches into shared main branches.	Cleaning up local private feature branches before opening a Pull Request.
Golden Rule	Safe for public shared branches.	NEVER rebase commits that have been pushed to a public/shared repository!

7. GITHUB COLLABORATION WORKFLOWS

7.1 The GitHub Flow (Step-by-Step)

Step 1 — Fork: Create personal remote copy of upstream repository on GitHub.

Step 2 — Clone & Branch: Clone fork locally and checkout a topic branch (git checkout -b topic-branch).

Step 3 — Commit & Push: Make code changes, commit, and push branch to fork (git push origin topic-branch).

Step 4 — Pull Request (PR): Open PR on GitHub targeting upstream repository main branch.

Step 5 — Peer & Faculty Review: Conduct discussion, address inline code comments, push additional fix commits.

Step 6 — Merge: Project owner approves and merges PR into upstream main branch.

7.2 Project Maintenance & Administration

- **Collaborator Access:** Settings -> Collaborators -> Grant read/write push access.
- **Security:** Enforce Two-Factor Authentication (2FA) for repository maintainers.
- **Webhooks & Automation:** Configure HTTP post payloads to trigger CI/CD pipelines on commit push events.

8. EXAM REVISION MODULE: Q&A

8.1 Part A: High-Yield Short Answer Questions & Answers

Q1: Define Software Engineering according to IEEE standard.

According to IEEE, Software Engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

Q2: Differentiate Verification and Validation.

Verification ensures that software correctly implements specific functionality ('Are we building the product right?') through reviews and inspections. Validation ensures software satisfies customer requirements ('Are we building the right product?') through functional and acceptance testing.

Q3: What are the 4 core values of the Agile Manifesto?

1. Individuals and interactions over processes and tools.
2. Working software over comprehensive documentation.
3. Customer collaboration over contract negotiation.
4. Responding to change over following a plan.

Q4: Enumerate the 5 Core XP Values.

The 5 Extreme Programming values are: Communication, Simplicity, Feedback, Courage, and Respect.

Q5: Differentiate 'git pull' and 'git fetch'.

git fetch retrieves remote commits and stores them in remote-tracking branches without altering local working code. git pull performs git fetch followed immediately by git merge to update the active branch. (*git pull = git fetch + git merge*).

Q6: Differentiate 'git diff' and 'git status'.

git status displays state summaries of working directory and staging index (modified, staged, untracked). git diff reveals line-by-line file differences across commits, staging, or working directory.

Q7: What is the 80% Rule in DSDM?

In DSDM, each iteration follows the 80% rule: only enough work is completed in an increment (80%) to facilitate movement to the next increment, leaving remaining details to be finalized as requirements evolve.

Q8: Explain the role of a Scrum Master.

The Scrum Master acts as a team leader who protects the development team from external distractions, enforces Scrum practices, and facilitates the Daily Scrum meeting.

8.2 Part B: Semester Examination Practice Questions

1. Compare the Waterfall Model and Agile Process Paradigm in terms of flexibility, customer involvement, risk management, and cost of change curve. (13 Marks)
2. Discuss Extreme Programming (XP) framework activities, user stories, project velocity, and 6 Industrial XP (IXP) enterprise practices. (13 Marks)
3. Explain the Scrum agile process model including Scrum roles, backlog management, sprint lifecycle, daily scrum mechanics, and scaling strategies. (13 Marks)
4. Detail the architecture of Distributed Version Control Systems (DVCS) and illustrate Git branching, merging, and rebasing mechanics with comparative evaluation. (13 Marks)
5. Describe the step-by-step GitHub Flow collaboration model from repository forking to Pull Request reviews and main branch merging. (13 Marks)